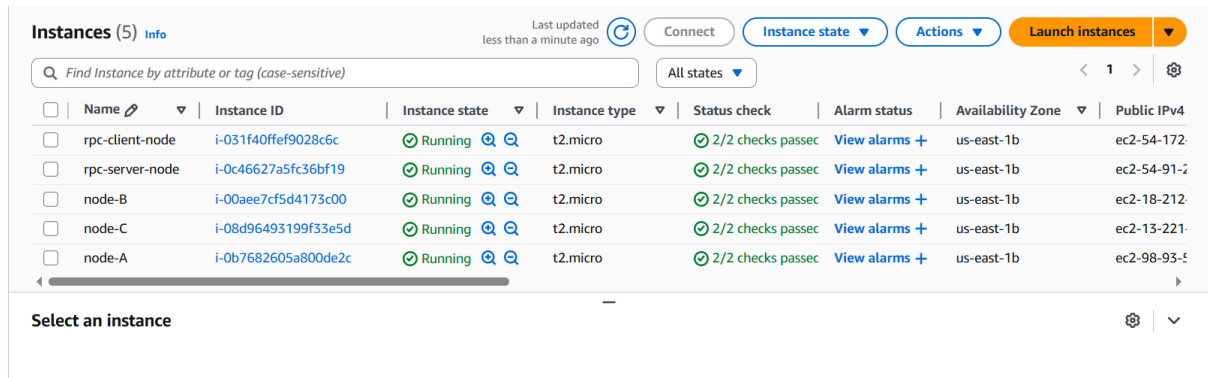


Zinetov Alikhan IT-2301 Assignment 2

GitHub Repo: <https://github.com/Zikaal/distributed-computing-lab2>

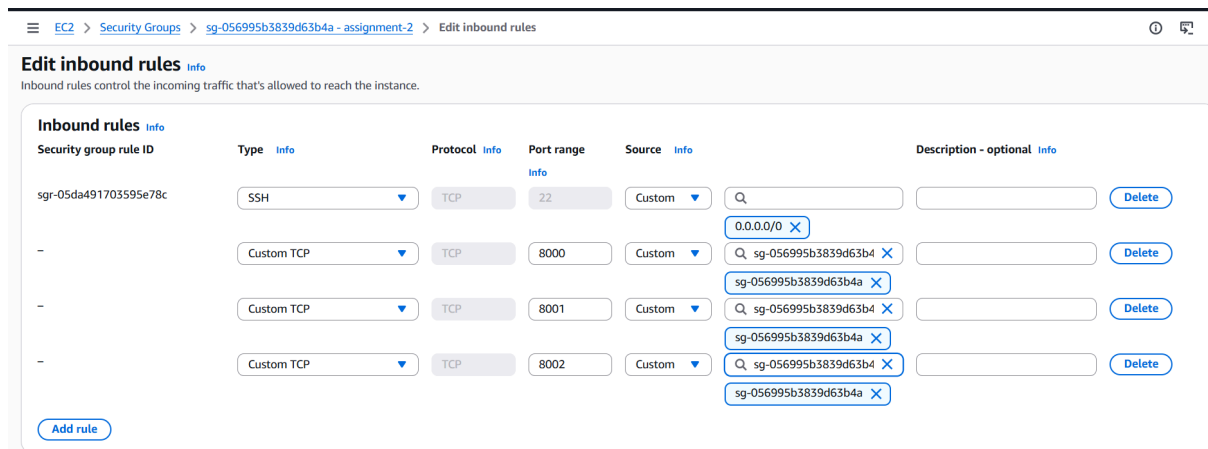
Three EC2 instances were created and reachable via SSH



The screenshot displays the AWS Management Console 'Instances' page. It shows five EC2 instances, all in a 'Running' state. The instances are named 'rpc-client-node', 'rpc-server-node', 'node-B', 'node-C', and 'node-A'. Each instance is a 't2.micro' type and is located in the 'us-east-1b' availability zone. The 'Status check' column indicates that all instances have passed their health checks (2/2 checks passed). The 'Public IPv4' addresses are listed for each instance.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
rpc-client-node	i-031f40ffe9028c6c	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-54-172...
rpc-server-node	i-0c46627a5fc36bf19	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-54-91-...
node-B	i-00aee7cf5d4173c00	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-18-212...
node-C	i-08d96493199f33e5d	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-13-221...
node-A	i-0b7682605a800de2c	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-98-93-...

Inbound rules allow your chosen ports (default in starter code: 8000/8001/8002).



The screenshot shows the 'Edit inbound rules' page for a security group. It lists four inbound rules, all of which are 'Custom TCP' rules. The first rule is for port 22 (SSH) with source '0.0.0.0/0'. The other three rules are for ports 8000, 8001, and 8002, all with source 'sg-056995b3839d63b4'. Each rule has a 'Delete' button next to it.

Security group rule ID	Type	Protocol	Port range	Source	Description - optional
sg-05da491703595e78c	SSH	TCP	22	Custom	0.0.0.0/0
-	Custom TCP	TCP	8000	Custom	sg-056995b3839d63b4
-	Custom TCP	TCP	8001	Custom	sg-056995b3839d63b4
-	Custom TCP	TCP	8002	Custom	sg-056995b3839d63b4

node-A:

```
PS D:\> cd D:\User\Desktop
PS D:\User\Desktop> ssh -i .\rpc-key.pem ubuntu@98.93.59.138
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1040-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Jan  2 22:00:22 UTC 2026

System load:  0.0               Processes:            101
Usage of /:   22.7% of 7.57GB   Users logged in:     0
Memory usage: 20%              IPv4 address for eth0: 172.31.23.252
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-23-252:~$
```

node-B:

```
ubuntu@ip-172-31-17-147: ~
PS C:\Users\User> cd D:\User\Desktop
PS D:\User\Desktop> ssh -i .\rpc-key.pem ubuntu@18.212.223.174
The authenticity of host '18.212.223.174 (18.212.223.174)' can't be established.
ECDSA key fingerprint is SHA256:XlMEqemQNfi9Sr0hNtdcgrLWUH8aZDhlGwlrOXRVOqw.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '18.212.223.174' (ECDSA) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1040-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Jan  2 22:01:21 UTC 2026

System load:  0.0               Processes:            100
Usage of /:   22.8% of 7.57GB   Users logged in:     0
Memory usage: 20%              IPv4 address for eth0: 172.31.17.147
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-17-147:~$
```

node-C:

```
PS C:\Users\User> cd D:\User\Desktop
PS D:\User\Desktop> ssh -i .\rpc-key.pem ubuntu@13.221.23.152
The authenticity of host '13.221.23.152 (13.221.23.152)' can't be established.
ECDSA key fingerprint is SHA256:KyJIBKbqR6tOllsrR0xlfUJHv1v3RyLmjCvXSqXoBPg.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '13.221.23.152' (ECDSA) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1040-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Jan  2 22:01:56 UTC 2026

System load:  0.0                       Processes:            101
Usage of /:   22.7% of 7.57GB           Users logged in:     0
Memory usage: 20%                      IPv4 address for eth0: 172.31.30.147
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-30-147:~$
```

On each instance, find its private IP:

node-A:

```
ubuntu@ip-172-31-23-252:~$ ip a | grep inet
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 172.31.23.252/20 metric 100 brd 172.31.31.255 scope global dynamic eth0
inet6 fe80::8ff:d0ff:fe49:37bf/64 scope link
ubuntu@ip-172-31-23-252:~$
```

node-B:

```
ubuntu@ip-172-31-17-147:~$ ip a | grep inet
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 172.31.17.147/20 metric 100 brd 172.31.31.255 scope global dynamic eth0
inet6 fe80::8ff:faff:fe18:38c3/64 scope link
ubuntu@ip-172-31-17-147:~$
```

node-C:

```
ubuntu@ip-172-31-30-147:~$ ip a | grep inet
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
    inet 172.31.30.147/20 metric 100 brd 172.31.31.255 scope global dynamic eth0
    inet6 fe80::8ff:caff:fe87:27c9/64 scope link
ubuntu@ip-172-31-30-147:~$
```

Upload `node.py` and `client.py` to each EC2 instance

node-A and similar to other nodes:

```
> PY
ubuntu@ip-172-31-23-252:~$ ls -la
total 36
drwxr-x--- 4 ubuntu ubuntu 4096 Jan  2 22:09 .
drwxr-xr-x 3 root root 4096 Jan  2 21:47 ..
-rw-r--r-- 1 ubuntu ubuntu 220 Jan  6 2022 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3771 Jan  6 2022 .bashrc
drwx----- 2 ubuntu ubuntu 4096 Jan  2 22:00 .cache
-rw-r--r-- 1 ubuntu ubuntu 807 Jan  6 2022 .profile
drwx----- 2 ubuntu ubuntu 4096 Jan  2 21:47 .ssh
-rw-r--r-- 1 ubuntu ubuntu 0 Jan  2 22:05 .sudo_as_admin_successful
-rw-rw-r-- 1 ubuntu ubuntu 768 Jan  2 22:09 client.py
-rw-rw-r-- 1 ubuntu ubuntu 3494 Jan  2 22:08 node.py
ubuntu@ip-172-31-23-252:~$
```

Run each node with a unique port:

```
ubuntu@ip-172-31-23-252:~$ python3 node.py --id A --port 8000 --peers http://172.31.17.147:8001,http://172.31.30.147:8002
Node A running on port 8000
```

```
ubuntu@ip-172-31-17-147:~$ python3 node.py --id B --port 8001 --peers http://172.31.23.252:8000,http://172.31.30.147:8002
Node B running on port 8001
```

```
connection to 172.31.23.252 8000 port [tcp/*] succeeded:
ubuntu@ip-172-31-30-147:~$ python3 node.py --id C --port 8002 --peers http://172.31.23.252:8000,http://172.31.17.147:8001
Node C running on port 8002
```

Verify Connectivity (Required)

node-A:

```
ubuntu@ip-172-31-23-252:~$ nc -vz 172.31.17.147 8001
Connection to 172.31.17.147 8001 port [tcp/*] succeeded!
ubuntu@ip-172-31-23-252:~$ nc -vz 172.31.30.147 8002
Connection to 172.31.30.147 8002 port [tcp/*] succeeded!
ubuntu@ip-172-31-23-252:~$
```

node-B:

```
ubuntu@ip-172-31-17-147:~$ nc -vz 172.31.30.147 8002
Connection to 172.31.30.147 8002 port [tcp/*] succeeded!
ubuntu@ip-172-31-17-147:~$ nc -vz 172.31.23.252 8000
Connection to 172.31.23.252 8000 port [tcp/*] succeeded!
ubuntu@ip-172-31-17-147:~$
```

node-C:

```
ubuntu@ip-172-31-30-147:~$ nc -vz 172.31.17.147 8001
Connection to 172.31.17.147 8001 port [tcp/*] succeeded!
ubuntu@ip-172-31-30-147:~$ nc -vz 172.31.23.252 8000
Connection to 172.31.23.252 8000 port [tcp/*] succeeded!
ubuntu@ip-172-31-30-147:~$
```

NODE PY:

```
import json
import time
import argparse
import threading
from http.server import HTTPServer, BaseHTTPRequestHandler
import urllib.request
import urllib.parse
```

class Node:

```
    def __init__(self, node_id, peers):
        self.node_id = node_id
        self.peers = peers
        self.clock = 0
        self.store = {} # key -> {"value":..., "ts":..., "origin":...}
        self.lock = threading.Lock()
```

Lamport: local event

```
    def tick(self):
        with self.lock:
            self.clock += 1
            return self.clock
```

Lamport: receive message with timestamp t

```
    def update_clock(self, t):
        with self.lock:
            self.clock = max(self.clock, int(t)) + 1
            return self.clock
```

```
    def get_clock(self):
```

```

with self.lock:
    return self.clock

def apply_update(self, key, value, ts, origin):
    ts = int(ts)
    with self.lock:
        cur = self.store.get(key)
        if (cur is None or
            ts > cur["ts"] or
            (ts == cur["ts"] and origin > cur["origin"])):
            self.store[key] = {"value": value, "ts": ts, "origin": origin}
            print(f"[{self.node_id}] APPLY {key}={value} ts={ts} from {origin} clock={self.clock}")

# Lamport: send message (tick + attach)
def replicate_to_peers(self, payload):
    for peer in self.peers:
        peer = peer.strip().rstrip("/")
        if not peer:
            continue

    # Scenario A: delay only A -> C (C uses port 8002)
    if self.node_id == "A" and peer.endswith(":8002"):
        print("[A] Artificial delay A -> C (2s)")
        time.sleep(2)

    for attempt in range(1, 4):
        try:
            data = json.dumps(payload).encode("utf-8")
            req = urllib.request.Request(
                peer + "/replicate",
                data=data,
                headers={"Content-Type": "application/json"},
                method="POST"
            )
            urllib.request.urlopen(req, timeout=5).read()
            print(f"[{self.node_id}] Replicated to {peer} attempt={attempt}")
            break
        except Exception as e:
            print(f"[{self.node_id}] Replication to {peer} failed attempt={attempt}: {repr(e)}")
            time.sleep(0.5 * attempt)

# Manual pull sync (for outage recovery demo)
def sync_from_peers(self):
    for peer in self.peers:
        peer = peer.strip().rstrip("/")
        if not peer:
            continue
    try:

```

```

raw = urllib.request.urlopen(peer + "/dump", timeout=5).read().decode("utf-8")
data = json.loads(raw) if raw else {}
peer_clock = int(data.get("clock", 0))
peer_store = data.get("store", {})

```

```

# receive event
self.update_clock(peer_clock)

```

```

# merge store using LWW rules
for k, v in peer_store.items():
    if isinstance(v, dict) and "value" in v and "ts" in v and "origin" in v:
        self.apply_update(k, v["value"], v["ts"], v["origin"])

```

```

print(f"[{self.node_id}] SYNC from {peer} OK")
except Exception as e:
    print(f"[{self.node_id}] SYNC from {peer} failed: {repr(e)}")

```

```

node = None

```

```

class Handler(BaseHTTPRequestHandler):

```

```

    def _send_json(self, code, obj):
        body = json.dumps(obj).encode("utf-8")
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

```

```

    def do_POST(self):
        global node
        length = int(self.headers.get("Content-Length", "0"))
        body = self.rfile.read(length).decode("utf-8")
        data = json.loads(body) if body else {}

```

```

        if self.path == "/put":
            key = data.get("key")
            value = data.get("value")
            if key is None:
                return self._send_json(400, {"error": "key is required"})

```

```

        # local event
        ts = node.tick()

```

```

        # apply locally (origin is this node)
        node.apply_update(key, value, ts, node.node_id)

```

```

        # send event: tick before sending and attach timestamp
        send_ts = node.tick()

```



```

        payload = {"key": key, "value": value, "ts": send_ts, "origin": node.node_id}

        threading.Thread(target=node.replicate_to_peers, args=(payload,),
daemon=True).start()
        return self._send_json(200, {"status": "OK", "key": key, "value": value, "ts": send_ts,
"origin": node.node_id})

    if self.path == "/replicate":
        key = data.get("key")
        value = data.get("value")
        ts = data.get("ts")
        origin = data.get("origin")

        if key is None or ts is None or origin is None:
            return self._send_json(400, {"error": "key, ts, origin required"})

        # receive event
        node.update_clock(ts)
        node.apply_update(key, value, ts, origin)
        return self._send_json(200, {"status": "OK"})

    if self.path == "/sync":
        # manual sync for Scenario C
        threading.Thread(target=node.sync_from_peers, daemon=True).start()
        return self._send_json(200, {"status": "SYNC_STARTED"})

    self._send_json(404, {"error": "not found"})

def do_GET(self):
    global node

    if self.path.startswith("/get"):
        qs = urllib.parse.urlparse(self.path).query
        params = urllib.parse.parse_qs(qs)
        key = params.get("key", [None])[0]
        if key is None:
            return self._send_json(400, {"error": "key query param required"})
        return self._send_json(200, {"key": key, "record": node.store.get(key)})

    if self.path == "/status":
        return self._send_json(200, {
            "node": node.node_id,
            "lamport": node.get_clock(),
            "store": node.store
        })

    if self.path == "/dump":
        # internal endpoint for sync

```

```

        return self._send_json(200, {
            "node": node.node_id,
            "clock": node.get_clock(),
            "store": node.store
        })

    self._send_json(404, {"error": "not found"})

def main():
    global node
    parser = argparse.ArgumentParser()
    parser.add_argument("--id", required=True)
    parser.add_argument("--port", type=int, required=True)
    parser.add_argument("--peers", required=True)
    args = parser.parse_args()

    peers = [p.strip() for p in args.peers.split(",") if p.strip()]
    node = Node(args.id, peers)

    server = HTTPServer(("0.0.0.0", args.port), Handler)
    print(f"Node {args.id} running on port {args.port}")
    print(f"Peers: {peers}")
    server.serve_forever()

if __name__ == "__main__":
    main()

```

CLIENT PY:

```

import argparse
import urllib.request
import urllib.parse
import json

parser = argparse.ArgumentParser()
parser.add_argument("--node", required=True)
parser.add_argument("cmd", choices=["put", "get", "status", "sync"])
parser.add_argument("key", nargs="?")
parser.add_argument("value", nargs="?")
args = parser.parse_args()

base = args.node.rstrip("/")

def pretty_print(resp_bytes):
    text = resp_bytes.decode("utf-8") if resp_bytes else ""
    try:

```

```

    obj = json.loads(text) if text else {}
    print(json.dumps(obj, indent=2))
except Exception:
    print(text)

if args.cmd == "put":
    if args.key is None or args.value is None:
        raise SystemExit("Usage: put <key> <value>")
    data = json.dumps({"key": args.key, "value": args.value}).encode("utf-8")
    req = urllib.request.Request(base + "/put", data=data, headers={"Content-Type":
"application/json"}, method="POST")
    pretty_print(urllib.request.urlopen(req).read())

elif args.cmd == "get":
    if args.key is None:
        raise SystemExit("Usage: get <key>")
    url = base + "/get?key=" + urllib.parse.quote(args.key)
    pretty_print(urllib.request.urlopen(url).read())

elif args.cmd == "status":
    pretty_print(urllib.request.urlopen(base + "/status").read())

elif args.cmd == "sync":
    req = urllib.request.Request(base + "/sync", data=b"{}", headers={"Content-Type":
"application/json"}, method="POST")
    pretty_print(urllib.request.urlopen(req).read())

```

Scenario A — Delay/Reorder (introduce artificial delay and show ordering effects): Delay replication from node A to node C only (starter code indicates where). Show that B may apply updates before C.

```

def replicate_to_peers(self, payload):
    for peer in self.peers:
        try:
            # Scenario A: artificial delay A -> C
            if self.node_id == "A" and peer.endswith(":8002"):
                print("Artificial delay A -> C")
                time.sleep(2)

            data = json.dumps(payload).encode()
            req = urllib.request.Request(peer + "/replicate", data=data, headers={'Content-Type': 'application/json'})
            urllib.request.urlopen(req, timeout=2)
        except Exception as e:
            print(f"[{self.node_id}] Replication to {peer} failed")

```

```
ubuntu@ip-172-31-23-252:~$ python3 client.py --node http://172.31.23.252:8000 put x 1
{
  "status": "OK",
  "key": "x",
  "value": "1",
  "ts": 4,
  "origin": "A"
}
ubuntu@ip-172-31-23-252:~$
```

```
ubuntu@ip-172-31-17-147:~$ python3 client.py --node http://172.31.17.147:8001 get x 1
{
  "key": "x",
  "record": {
    "value": "1",
    "ts": 4,
    "origin": "A"
  }
}
ubuntu@ip-172-31-17-147:~$
```

```
ubuntu@ip-172-31-30-147:~$ python3 client.py --node http://172.31.30.147:8002 get x 1
{
  "key": "x",
  "record": null
}
ubuntu@ip-172-31-30-147:~$ python3 client.py --node http://172.31.30.147:8002 get x 1
{
  "key": "x",
  "record": {
    "value": "1",
    "ts": 4,
    "origin": "A"
  }
}
ubuntu@ip-172-31-30-147:~$
```

In a distributed system, messages do not always arrive in the order in which they are sent because of network delays and routing differences. In this experiment, replication from Node A to Node C was deliberately delayed, while Node B received updates immediately. As a result, Node B reflected the new value earlier than Node C, even though both updates originated from the same operation. Once the delayed message reached Node C, it used the Lamport timestamp attached to the update to correctly apply the change and reach the same state as the other nodes. This shows that Lamport clocks preserve causal ordering and guarantee that delayed messages are applied correctly.

Scenario B — Concurrent Writes (concurrent updates to the same key from two nodes): Send PUT x=1 to node A and PUT x=2 to node B nearly simultaneously. Show LWW convergence and explain concurrency.

```
ubuntu@ip-172-31-23-252:~$ python3 client.py --node http://172.31.23.252:8000 put x 1
{
  "status": "OK",
  "key": "x",
  "value": "1",
  "ts": 6,
  "origin": "A"
}
ubuntu@ip-172-31-23-252:~$ _
```

```
ubuntu@ip-172-31-17-147:~$ python3 client.py --node http://172.31.17.147:8001 put x 2
{
  "status": "OK",
  "key": "x",
  "value": "2",
  "ts": 9,
  "origin": "B"
}
ubuntu@ip-172-31-17-147:~$ _
```

```
ubuntu@ip-172-31-30-147:~$ python3 client.py --node http://172.31.30.147:8002 get x
{
  "key": "x",
  "record": {
    "value": "2",
    "ts": 9,
    "origin": "B"
  }
}
ubuntu@ip-172-31-30-147:~$ _
```

When two nodes update the same key at nearly the same time, neither node is aware of the other's operation, so both updates are considered concurrent. In this case, both updates are assigned Lamport timestamps and propagated through the system. If the timestamps differ, the larger one wins. If the timestamps are equal, the node ID is used as a deterministic tie-breaker. This Last-Writer-Wins rule ensures that all nodes resolve the conflict in the same way, so the system converges to a single consistent value for the key despite the concurrent updates.

Scenario C — Temporary Outage: Stop node C, perform updates on A/B, restart C, and show eventual convergence via /status.

```
ubuntu@ip-172-31-30-147:~$ python3 node.py --id C --port 8002 --peers http://172.31.23.252:8000,http://172.31.17.147:8001
Node C running on port 8002
Peers: ['http://172.31.23.252:8000', 'http://172.31.17.147:8001']
[C] APPLY x=2 ts=3 from B clock=4
172.31.17.147 - - [03/Jan/2026 15:58:31] "POST /replicate HTTP/1.1" 200 -
172.31.23.252 - - [03/Jan/2026 15:58:31] "POST /replicate HTTP/1.1" 200 -
172.31.23.252 - - [03/Jan/2026 15:59:00] "GET /get?key=x HTTP/1.1" 200 -
172.31.23.252 - - [03/Jan/2026 15:59:12] "GET /get?key=x HTTP/1.1" 200 -
^CTraceback (most recent call last):
  File "/home/ubuntu/node.py", line 136, in <module>
    main()
  File "/home/ubuntu/node.py", line 133, in main
    server.serve_forever()
  File "/usr/lib/python3.10/socketserver.py", line 232, in serve_forever
    ready = selector.select(poll_interval)
  File "/usr/lib/python3.10/selectors.py", line 416, in select
    fd_event_list = self._selector.poll(timeout)
KeyboardInterrupt
ubuntu@ip-172-31-30-147:~$
```

```
ubuntu@ip-172-31-23-252:~$ python3 client.py --node http://172.31.23.252:8000 put y 100
{
  "status": "OK",
  "key": "y",
  "value": "100",
  "ts": 12,
  "origin": "A"
}
ubuntu@ip-172-31-23-252:~$ python3 client.py --node http://172.31.23.252:8000 put z 200
{
  "status": "OK",
  "key": "z",
  "value": "200",
  "ts": 14,
  "origin": "A"
}
ubuntu@ip-172-31-23-252:~$
```

```
ubuntu@ip-172-31-30-147:~$ python3 node.py --id C --port 8002 --peers http://172.31.23.252:8000,http://172.31.17.147:8001
Node C running on port 8002
Peers: ['http://172.31.23.252:8000', 'http://172.31.17.147:8001']
-
```

```
ubuntu@ip-172-31-30-147:~$ python3 client.py --node http://172.31.30.147:8002 sync
{
  "status": "SYNC_STARTED"
}
ubuntu@ip-172-31-30-147:~$ python3 client.py --node http://172.31.30.147:8002 status
{
  "node": "C",
  "lamport": 16,
  "store": {
    "x": {
      "value": "2",
      "ts": 9,
      "origin": "B"
    },
    "y": {
      "value": "100",
      "ts": 12,
      "origin": "A"
    },
    "z": {
      "value": "200",
      "ts": 14,
      "origin": "A"
    }
  }
}
ubuntu@ip-172-31-30-147:~$
```

When a node goes offline, it misses all updates that occur during that period. In this scenario, Node C was stopped while Nodes A and B continued processing updates. After Node C restarted, it synchronized with its peers and received all missing updates. Using Lamport timestamps and the LWW rule, it correctly merged the received data with its local state. After synchronization, Node C contained the same values as the other nodes, demonstrating eventual consistency: temporary failures do not permanently break system correctness, and all nodes converge once communication is restored.